

04 - Enterprise Tech Stack for Products & Client Projects

LIONGATE SARL - Engineering Standards

Version 1.0 - Initial release

Table of Contents

1. [Overview & Philosophy](#)
2. [Stack Categories by Project Type](#)
3. [Backend Stack](#)
4. [Frontend Stack](#)
5. [Database Stack](#)
6. [DevOps Stack](#)
7. [Testing Stack](#)
8. [Observability Stack](#)
9. [Security Stack](#)
10. [API Design Standards](#)
11. [CI/CD Stack](#)
12. [File & Repository Structure Standards](#)
13. [Environment Management Standards](#)
14. [Versioning Strategy](#)
15. [Code Quality Standards](#)
16. [Decision Matrix by Project Type](#)

Overview & Philosophy

Core Principle

LIONGATE SARL standardizes on a **preferred default stack** for most projects and documents approved alternatives for specific use cases. Choosing a stack outside of this document requires a

documented Architecture Decision Record (ADR).

Four Project Categories

Category	Definition	Stack priority
Internal product	LIONGATE's own tools (ERP, dashboards)	Standard or Odoo-based
Client project	Software built for a specific client	Standard stack (customized)
MVP / Prototype	Fast validation with minimal investment	Lightweight stack
Enterprise-grade	High availability, SLA-bound, large scale	Full stack with HA

Design Goals

- **Reduce cognitive overhead** - one stack to master, not ten
- **Enable team rotation** - any engineer can work on any project
- **Production-ready by default** - not bolted on later
- **Documented trade-offs** - not "it depends" without explanation

Stack Categories by Project Type

Component	Internal Product	Client Project	MVP	Enterprise
Backend	NestJS or FastAPI	NestJS or FastAPI	FastAPI	NestJS
Frontend	Next.js or React	Next.js	Next.js	Next.js
Database	PostgreSQL	PostgreSQL	PostgreSQL	PostgreSQL + Redis
API style	REST or GraphQL	REST	REST	REST + WebSocket
Auth	JWT + refresh	JWT + refresh	JWT	OAuth2 + RBAC
Containerization	Docker	Docker	Docker	Docker

Component	Internal Product	Client Project	MVP	Enterprise
Orchestration	Docker Compose	Docker Compose	Docker Compose	Compose or K8s
CI/CD	GitHub Actions	GitHub Actions	GitHub Actions	GitHub Actions
Monitoring	Grafana stack	Grafana stack	UptimeRobot	Full Grafana stack
Tests	Unit + Integration	Unit + E2E	Unit	Unit + Integration + E2E + Load

Backend Stack

Primary Recommendation: NestJS (TypeScript)

Criterion	NestJS
Language	TypeScript (Node.js)
Architecture	Modular, injectable, opinionated
API style	REST (default) or GraphQL (optional)
ORM	TypeORM or Prisma
Auth	Passport.js + JWT
Validation	class-validator + class-transformer
Background jobs	Bull (Redis-based queue)
Testing	Jest
Documentation	Swagger (auto-generated via decorators)

Why NestJS:

NestJS enforces clean architecture out of the box (modules, controllers, services). It is strongly typed, has excellent documentation, and scales from MVP to enterprise without rewrite. Most backend engineers familiar with Angular or Spring will recognize the patterns immediately.

Alternative Recommendation: FastAPI (Python)

Use FastAPI when:

- The team is primarily Python-experienced
- The project involves ML/AI integrations
- Rapid scripting and data processing is core to the backend
- The project will closely integrate with Odoo (shared Python ecosystem)

Criterion	FastAPI
Language	Python 3.11+
Architecture	Function-based routes, dependency injection
ORM	SQLAlchemy 2.x + Alembic
Auth	python-jose (JWT)
Validation	Pydantic v2
Background jobs	Celery + Redis
Testing	Pytest + httpx
Documentation	Auto-generated OpenAPI

Backend Project Structure (NestJS)

```
src/  
├── app.module.ts  
├── main.ts  
├── config/  
│   └── configuration.ts  
├── common/  
│   ├── decorators/  
│   ├── filters/  
│   ├── guards/  
│   ├── interceptors/  
│   └── pipes/  
├── modules/  
│   ├── auth/  
│   │   ├── auth.module.ts  
│   │   ├── auth.controller.ts  
│   │   ├── auth.service.ts  
│   │   └── strategies/  
│   ├── users/  
│   │   ├── users.module.ts  
│   │   ├── users.controller.ts  
│   │   ├── users.service.ts  
│   │   ├── users.repository.ts  
│   │   └── dto/  
│   └── [feature]/  
│       ├── [feature].module.ts  
│       ├── [feature].controller.ts  
│       ├── [feature].service.ts  
│       └── dto/  
└── database/  
    ├── migrations/  
    └── seeds/
```

Authentication & Authorization Standards

Authentication flow

1. User POSTs credentials → /auth/login
2. Server validates, returns:
 - access_token (JWT, 15 min expiry)
 - refresh_token (JWT, 7 days expiry, stored in httpOnly cookie)
3. Client includes Bearer token in Authorization header
4. Server validates token on every protected route
5. Client uses /auth/refresh to get new access_token before expiry

Authorization model

- Role-based (RBAC): admin, manager, user, viewer
- Route-level guards: @Roles('admin')
- Field-level permissions: for sensitive data
- All permission checks logged to audit_log table

Error Handling Standards

```
// Standard error response shape
{
  "statusCode": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "details": [
    { "field": "email", "constraint": "isEmail" }
  ],
  "timestamp": "2024-01-15T10:30:00Z",
  "path": "/api/v1/users"
}
```

```
// Never expose:
// – Stack traces in production
// – Internal database errors
// – File system paths
// – Environment variables
```

Logging Standards

```
// Log levels: error > warn > log > debug > verbose
// Production: log + warn + error only
// Staging: debug
// Dev: verbose
```

```
// Required log fields
{
  level: "error",
  message: "Database connection failed",
  service: "UserService",
  method: "findById",
  userId: "uuid",    // if available
  requestId: "uuid",
  timestamp: "ISO8601",
  duration: 234      // ms
}
```

```
// Never log:
// - Passwords
// - Tokens
// - Credit card numbers
// - PII beyond user ID
```

Background Jobs (Bull / Celery)

```
// Job queue naming convention
// queue: {domain}.{action}
// Examples:
//   "email.send-welcome"
//   "report.generate-monthly"
//   "data.sync-erp"

// Job retry policy
{
  attempts: 3,
  backoff: { type: "exponential", delay: 2000 },
  removeOnComplete: 100,    // keep last 100 completed
  removeOnFail: 500         // keep last 500 failed for inspection
}
```

Frontend Stack

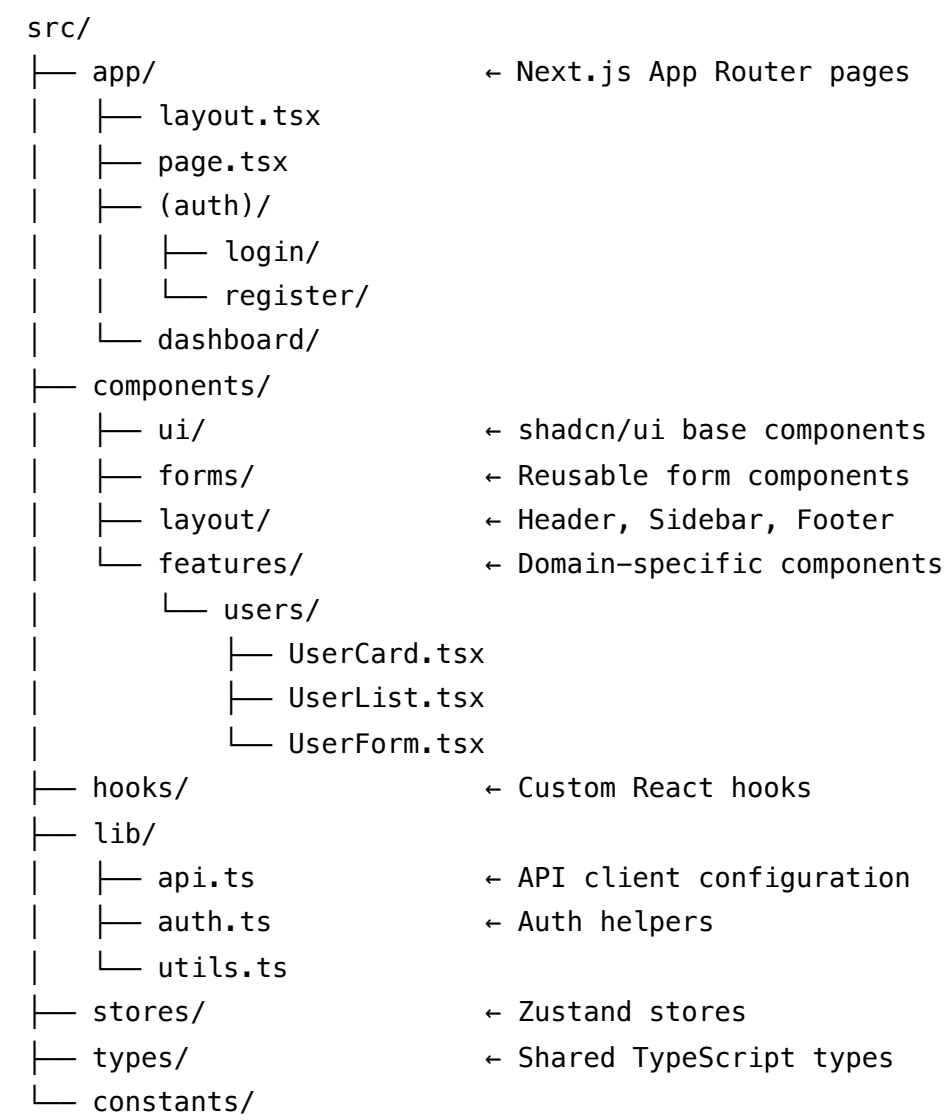
Primary Recommendation: Next.js (React / TypeScript)

Criterion	Next.js
Language	TypeScript
Rendering	SSR, SSG, CSR, ISR - per page
Styling	Tailwind CSS + shadcn/ui or Ant Design
State management	Zustand (global) + React Query (server state)
Forms	React Hook Form + Zod
HTTP client	Axios or fetch + React Query
Testing	Jest + React Testing Library + Playwright
Build	Next.js built-in (Turbopack or Webpack)
Deployment	Docker static export or Vercel (lightweight clients)

Why Next.js:

Next.js is the most production-ready React framework. It handles routing, SSR/SSG, API routes, image optimization, and bundle splitting out of the box. The ecosystem is large, hiring is easier, and it integrates cleanly with any backend API.

Component Architecture



State Management Rules

State type	Tool	Example
Server/async state	React Query (useQuery , useMutation)	User list, product data
Global UI state	Zustand	Auth user, sidebar state, toast notifications
Local component state	useState	Form input, modal open/close
URL state	useSearchParams	Filters, pagination

Do NOT use Redux unless there is a documented reason. Zustand covers 95% of cases with far less boilerplate.

Forms Standard

```
// Always use React Hook Form + Zod
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

const schema = z.object({
  email: z.string().email("Invalid email"),
  password: z.string().min(8, "Minimum 8 characters"),
});

type FormData = z.infer<typeof schema>;

const form = useForm<FormData>({ resolver: zodResolver(schema) });
```

Data Fetching Standard

```
// Always React Query for server data
const { data, isLoading, error } = useQuery({
  queryKey: ["users", filters],
  queryFn: () => api.get("/users", { params: filters }),
  staleTime: 5 * 60 * 1000, // 5 minutes
});

// Mutations
const mutation = useMutation({
  mutationFn: (data) => api.post("/users", data),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["users"] });
    toast.success("User created");
  },
});
```

Database Stack

Primary: PostgreSQL 16

PostgreSQL is the default database for all LIONGATE projects.

Use case	PostgreSQL feature
Relational data	Tables, joins, foreign keys
JSON/semi-structured	jsonb column type
Full-text search	tsvector + GIN index
Geospatial	PostGIS extension
Audit trail	Triggers + audit table
Time series	TimescaleDB extension (optional)

Secondary: Redis

Redis is used alongside PostgreSQL for:

- Session storage
- JWT blacklist / token revocation
- Bull job queues
- Rate limiting counters
- Short-lived cache (API responses < 5 min TTL)

Migration Strategy

Tool: Alembic (Python) or TypeORM Migrations (NestJS)

Rules:

1. NEVER modify a migration file after it has been applied in any environment
2. Every schema change requires a new migration file
3. Migration files are named: YYYYMMDDHHMMSS_description.ts
4. Destructive migrations (DROP TABLE, DROP COLUMN) require:
 - a backward-compatible transition period (soft delete or rename first)
 - explicit review and approval
5. Migrations are run automatically on deployment (after backup)
6. Rollback migrations must be written when possible

Backup & Restore

Backup

```
pg_dump -h $DB_HOST -U $DB_USER -d $DB_NAME -Fc -f backup_$(date +%Y%m%d).dump
```

Restore

```
pg_restore -h $DB_HOST -U $DB_USER -d $DB_NAME backup_20240115.dump
```

Schedule: Daily at 02:00, 7-day retention

Storage: Hetzner Storage Box

Restore must be tested monthly

Schema Management Standards

```
-- Table naming: snake_case plural
CREATE TABLE user_profiles (...)

-- Column naming: snake_case
created_at TIMESTAMPTZ DEFAULT NOW()
updated_at TIMESTAMPTZ

-- Every table must have:
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

-- Soft deletes (preferred over hard deletes)
deleted_at TIMESTAMPTZ DEFAULT NULL

-- Foreign keys: always with ON DELETE behavior documented
user_id UUID REFERENCES users(id) ON DELETE CASCADE
```

DevOps Stack

Containerization

Tool	Purpose	Version
Docker	Container runtime	24+
Docker Compose	Local + production stack definition	v2
Docker Hub or GHCR	Container registry	-

Every service must have:

- A Dockerfile with multi-stage build
- .dockerignore file
- Health check defined in Dockerfile or Compose

Dockerfile Standard (NestJS example)

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production=false
COPY . .
RUN npm run build

# Stage 2: Production
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nestjs

COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package.json ./

USER nestjs

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD wget -q0- http://localhost:3000/health || exit 1

CMD ["node", "dist/main.js"]
```

Orchestration

Scale	Tool
Single server	Docker Compose
Multi-server (manual)	Docker Compose per server + Nginx LB
Large scale (if needed)	Docker Swarm or Kubernetes (future)

Default: Docker Compose. Move to orchestration only when genuinely needed.

Infrastructure as Code

Tools:

- Terraform: Provision Hetzner resources (servers, networks, firewalls, volumes)
- Ansible: Configure servers (install packages, set up services)
- GitHub Actions: CI/CD pipelines

Repository structure:

```
infrastructure/  
├─ terraform/  
│   ├── modules/  
│   └─ environments/  
├─ ansible/  
│   ├── playbooks/  
│   └─ inventory/  
└─ scripts/
```

Secrets Management

Environment	Method
Local dev	.env file (never committed)
CI/CD	GitHub Actions Secrets
Production	.env on server (600 permissions) + vault in future
Docker	--env-file or Docker secrets

Rules:

1. .env files are NEVER committed to git - .gitignore is mandatory
2. .env.example with all keys and no values is committed
3. Secrets are rotated every 90 days (minimum)
4. No secret is logged under any circumstances

Testing Stack

Testing Pyramid

```
      /\
     /E2E\      ← Playwright / Cypress (10–20 tests, critical paths)
    /-----\
   /Integr. \   ← Jest + Supertest (50–100 tests, API flows)
  /-----\
 /  Unit Tests \ ← Jest (200+ tests, business logic)
/-----\
```

Unit Testing (Jest)

```
// Test file: [module].spec.ts
// Co-located with source file
```

```
// What to unit test:
// - Service methods
// - Utility functions
// - Transformers / mappers
// - Validators
```

```
// What NOT to unit test:
// - Simple DTOs
// - Config files
// - Auto-generated code
```

```
// Coverage targets:
// - Services: 80%+
// - Utils: 90%+
// - Controllers: 60%+ (prefer integration tests here)
```


Integration Testing

```
// Test actual HTTP endpoints with in-memory or test DB
// Use: Jest + Supertest (NestJS) or Pytest + httpx (FastAPI)

// Test database: separate test DB, reset between test suites
// Environment: TEST_DATABASE_URL env var

// What to integration test:
// - Full API request/response cycle
// - Auth middleware
// - DB reads/writes
// - Business flows (create order → check inventory)
```

E2E Testing (Playwright)

```
// Test file: tests/e2e/[flow].spec.ts

// Coverage: critical user journeys only
// Examples:
//   - User can log in and see dashboard
//   - User can create a new record
//   - Admin can manage users
//   - Payment flow completes

// Run: CI on staging before production deploy
// Never: Block dev workflow (run separately)
```

Linting & Formatting

Tool	Config file	Purpose
ESLint	.eslintrc.js	TypeScript linting
Prettier	.prettierrc	Code formatting
pylint / ruff	pyproject.toml	Python linting
black	pyproject.toml	Python formatting
Husky + lint-staged	.husky/	Pre-commit hooks

Pre-commit hooks run:

1. `eslint --fix` or `ruff check --fix`
2. `prettier --write`
3. `jest --testPathPattern` (tests related to changed files)

Observability Stack

Standard Observability Stack

Metrics: Prometheus → Grafana

Logs: Loki → Grafana

Traces: OpenTelemetry → Tempo → Grafana (optional, advanced)

Alerts: Alertmanager → Email / Slack

Uptime: UptimeRobot (external)

Deployment (All on monitoring server - CX22)

```
# docker-compose.monitoring.yml
services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    ports:
      - "127.0.0.1:9090:9090"

  grafana:
    image: grafana/grafana:latest
    volumes:
      - grafana_data:/var/lib/grafana
    ports:
      - "127.0.0.1:3000:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD}

  loki:
    image: grafana/loki:latest
    ports:
      - "127.0.0.1:3100:3100"

  alertmanager:
    image: prom/alertmanager:latest
    volumes:
      - ./alertmanager.yml:/etc/alertmanager/alertmanager.yml
```

Application Metrics

Every backend service must expose a `/metrics` endpoint (Prometheus format):

```
// NestJS: use @willsoo/nestjs-prometheus
// Key metrics to expose:
// - http_requests_total (by method, route, status)
// - http_request_duration_seconds (histogram)
// - active_connections
// - job_queue_length
// - database_query_duration_seconds
```

Standard Dashboards

Dashboard	Panels
Server overview	CPU, RAM, disk, network per server
Application	Request rate, error rate, latency (P50/P95/P99)
Database	Query rate, connections, slow queries
Business metrics	Active users, key domain KPIs
Backup status	Last backup time, backup size

Security Stack

Security Layers

Layer	Tool / Practice
Secrets management	.env + GitHub Secrets
Dependency scanning	npm audit / pip-audit in CI
SAST	ESLint security plugin / Bandit (Python)
Container scanning	Trivy (in CI pipeline)
SSL/TLS	Let's Encrypt, TLS 1.2+ only
Firewall	Hetzner Firewall + UFW
Intrusion detection	Fail2ban

Layer	Tool / Practice
Audit logging	DB-level audit table per entity
RBAC	Built into every API (no open endpoints)
CORS	Explicit allowlist of origins
Rate limiting	NestJS ThrottlerModule / fastapi-limiter
Input validation	Zod (TS) / Pydantic (Python) on every endpoint
SQL injection	Parameterized queries via ORM - never raw SQL with interpolation
XSS	React escaping + CSP headers
CSRF	SameSite cookies + CSRF token for forms

Security CI Checks (mandatory)

```
# .github/workflows/security.yml
jobs:
  dependency-audit:
    steps:
      - run: npm audit --audit-level=high
  container-scan:
    steps:
      - uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${ env.IMAGE }
          severity: CRITICAL,HIGH
          exit-code: 1
  secret-scan:
    steps:
      - uses: trufflesecurity/trufflehog@main
```

Audit Logging

```
-- Every production system must have an audit_log table
CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID,
  action VARCHAR(50) NOT NULL,    -- CREATE, UPDATE, DELETE, LOGIN, EXPORT
  resource_type VARCHAR(100),
  resource_id UUID,
  changes JSONB,                  -- {before: {...}, after: {...}}
  ip_address INET,
  user_agent TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Retain audit logs for minimum 1 year
-- Do NOT delete audit logs - archive to cold storage after 1 year
```

API Design Standards

REST API Conventions

Base URL: `https://api.clientname.com/api/v1`

Versioning: URL-based (/v1, /v2) – never break v1 while v2 exists

Resources: plural nouns

GET `/api/v1/users`

GET `/api/v1/users/:id`

POST `/api/v1/users`

PATCH `/api/v1/users/:id` ← prefer PATCH over PUT

DELETE `/api/v1/users/:id`

Nested:

GET `/api/v1/users/:id/orders`

POST `/api/v1/users/:id/orders`

Filtering: `/api/v1/users?role=admin&status=active`

Sorting: `/api/v1/users?sort=createdAt&order=desc`

Pagination: `/api/v1/users?page=1&limit=20`

Search: `/api/v1/users?q=john`

Standard Response Envelope

```
// Success (single item)
{
  "data": { ... },
  "meta": { "requestId": "uuid" }
}

// Success (list)
{
  "data": [ ... ],
  "meta": {
    "total": 150,
    "page": 1,
    "limit": 20,
    "totalPages": 8
  }
}

// Error
{
  "statusCode": 422,
  "error": "Unprocessable Entity",
  "message": "Validation failed",
  "details": [ ... ],
  "timestamp": "2024-01-15T10:30:00Z"
}
```

API Documentation

- Every API is auto-documented via Swagger (NestJS) or OpenAPI (FastAPI)
- Swagger UI available at `/api/docs` in staging (not production)
- All endpoints must have: summary, description, request schema, response schemas, error responses documented

CI/CD Stack

GitHub Actions - Standard Workflow

```
# .github/workflows/main.yml

name: CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint
      - run: npm run format:check

  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test_db
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run test:cov
      - uses: codecov/codecov-action@v4

  build:
    needs: [lint, test]
    runs-on: ubuntu-latest
```

steps:

- uses: actions/checkout@v4
 - uses: docker/build-push-action@v5
- with:
- push: `${{ github.ref == 'refs/heads/main' }}`
 - tags: `ghcr.io/liongate/${{ env.APP_NAME }}:${{ github.sha }}`

deploy-staging:

needs: build

if: `github.ref == 'refs/heads/develop'`

runs-on: ubuntu-latest

steps:

- name: Deploy to staging
- uses: appleboy/ssh-action@v1

with:

- host: `${{ secrets.STAGING_HOST }}`
- username: deploy
- key: `${{ secrets.STAGING_SSH_KEY }}`
- script: |
 - `cd /opt/app`
 - `docker compose pull`
 - `docker compose up -d`
 - `docker compose exec app npm run db:migrate`

deploy-production:

needs: build

if: `github.ref == 'refs/heads/main'`

environment: production # ← requires manual approval

runs-on: ubuntu-latest

steps:

- name: Deploy to production
- uses: appleboy/ssh-action@v1

with:

- host: `${{ secrets.PROD_HOST }}`
- username: deploy
- key: `${{ secrets.PROD_SSH_KEY }}`
- script: |
 - `cd /opt/app`
 - `docker compose pull`
 - `docker compose up -d --no-deps app`
 - `docker compose exec app npm run db:migrate`

Branch Strategy

main	← Production deployments (tag: vX.Y.Z)
develop	← Staging deployments (auto-deploy)
feature/*	← Feature branches (PR → develop)
fix/*	← Bug fixes (PR → develop or main for hotfix)
release/*	← Release branches (for version prep)
hotfix/*	← Emergency production fixes (PR → main + develop)

Deployment Environments per Branch

Branch	Environment	Deploy	Approval
feature/*	None	No	-
develop	Staging	Auto	None
main	Production	Auto	Manual gate
hotfix/*	Production	Manual	None

File & Repository Structure Standards

Monorepo vs Polyrepo

Approach	When to use
Polyrepo (separate repos)	Default - one repo per service/project
Monorepo	Only when frontend + backend are tightly coupled and always deployed together

Mandatory Repository Files

Every repository must contain:

```
/
├─ README.md           ← Project description, setup, run commands
├─ CONTRIBUTING.md     ← How to contribute (branch, PR, review)
├─ CHANGELOG.md        ← Version history (auto-generated preferred)
├─ .env.example        ← All env vars documented, no values
├─ .gitignore
├─ .eslintrc.js / pyproject.toml
├─ .prettierrc
├─ docker-compose.yml  ← Local development stack
├─ Dockerfile
├─ Makefile            ← Common commands (make dev, make test, make build)
└─ docs/
    └─ architecture.md
        └─ api.md
```

Makefile Standard

.PHONY: dev test build deploy-staging

```
dev:          ## Start local development
    docker compose up -d

test:         ## Run all tests
    npm test

build:        ## Build Docker image
    docker build -t $(APP_NAME):local .

lint:         ## Run linter
    npm run lint

migrate:      ## Run database migrations
    npm run db:migrate

seed:         ## Seed database
    npm run db:seed

logs:         ## Tail application logs
    docker compose logs -f app
```

Environment Management Standards

Environment Variables

```
# Naming convention: SCREAMING_SNAKE_CASE
# Prefix by category:
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD
JWT_SECRET, JWT_EXPIRY
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD
REDIS_URL
STORAGE_BUCKET
APP_URL
NODE_ENV                ← development | staging | production
LOG_LEVEL                ← debug | info | warn | error
```

Environment Validation (startup)

```
// NestJS: validate env on startup
import { plainToInstance } from "class-transformer";
import { IsNotEmpty, IsString, validateSync } from "class-validator";

class EnvironmentVariables {
  @IsNotEmpty() DB_HOST: string;
  @IsNotEmpty() DB_PASSWORD: string;
  @IsNotEmpty() JWT_SECRET: string;
}

export function validate(config: Record<string, unknown>) {
  const validatedConfig = plainToInstance(EnvironmentVariables, config);
  const errors = validateSync(validatedConfig);
  if (errors.length > 0) {
    throw new Error(`Config validation failed: ${errors}`);
  }
  return validatedConfig;
}

// App will crash on startup if required env vars are missing – this is intentional
```

Versioning Strategy

Semantic Versioning

All projects follow **SemVer** (MAJOR.MINOR.PATCH):

Change	Version bump	Example
Breaking API change	MAJOR	1.x.x → 2.0.0
New feature, backward compatible	MINOR	1.2.x → 1.3.0
Bug fix, no new feature	PATCH	1.2.3 → 1.2.4

Release Tagging

```
# Tag format: vMAJOR.MINOR.PATCH
git tag v1.3.0 -m "Release v1.3.0: Add user management API"
git push origin v1.3.0

# GitHub release: auto-generate from tag
# Changelog: auto-generate with conventional commits
```

Conventional Commits

All commit messages follow the Conventional Commits specification:

```
feat: add user export endpoint
fix: resolve pagination off-by-one error
docs: update API documentation
chore: upgrade dependencies
refactor: extract auth middleware
test: add integration tests for user service
perf: add index on users.email column
ci: add security scanning workflow
breaking change: rename /users to /accounts
```

Code Quality Standards

Pull Request Rules

1. **No direct commits to main or develop** - always via PR
2. **Minimum 1 reviewer approval** required
3. **CI must pass** before merge
4. **PR description** must include: what, why, how to test
5. **PRs are kept small** - max 400 lines changed (aim for 200)
6. **PRs must be squash-merged** to keep main history clean

Code Review Standards

Reviewers check for:

- ☐ Business logic correctness
- ☐ Error handling completeness
- ☐ Security: no secrets exposed, input validated, RBAC applied
- ☐ Test coverage: new code has tests
- ☐ Database: migrations present, indexes considered
- ☐ Logging: appropriate log levels, no PII logged
- ☐ API: follows REST conventions, documented in Swagger
- ☐ Performance: no N+1 queries, no synchronous heavy operations in request path

Definition of "Production-Ready" Code

A feature is production-ready when:

- ☐ Unit tests written and passing
- ☐ Integration tests cover the main flow
- ☐ API documented in Swagger
- ☐ Error handling covers all failure modes
- ☐ Logging added at appropriate levels
- ☐ Migration reviewed and tested
- ☐ Security review passed (RBAC, input validation)
- ☐ Code reviewed and approved
- ☐ Deployed to staging and tested
- ☐ Product owner or tech lead signed off

Decision Matrix by Project Type

Decision	Internal Product	Client Project	MVP	Enterprise
TypeScript strict mode	Yes	Yes	Yes	Yes
Code coverage minimum	70%	80%	50%	85%
E2E tests required	No	Yes	No	Yes
Load testing required	No	Optional	No	Yes
Audit logging	Yes	Yes	No	Yes
Redis required	No	Optional	No	Yes
Separate monitoring server	No (share)	No	No	Yes
SLA defined	No	Client-specific	No	Yes
Disaster recovery drill	Quarterly	Quarterly	None	Monthly

Document owner: LIONGATE SARL Engineering Team

Last updated: See Git history

Next review: Quarterly or when a new major technology is adopted